

T_After notes

Figuring out what the Value of T_After actually represents

After figuring out what the T_After I collected actually represents, I found I was misguided previously about what it represents.

The code was:

```
Float pr = T_maj[0] * mp.sigma_s[0];
beta *= T_maj * mp.sigma_s / pr;
r_u *= T_maj * mp.sigma_s / pr;
Listing 1: Spectral MIS sampling
```

And then later..

```
if (!scattered && beta)
{
    beta *= T_maj / T_maj[0];
    r_u *= T_maj / T_maj[0];
    r_l *= T_maj / T_maj[0];
}
```

And the calculation I performed at the end is:

$$T_{\text{ray segment}} = \frac{\beta_{\text{after}}}{\beta_{\text{before}}}$$

So what I thought was the transmission just for the ray segment that was sampled, it was actually something different. Due to the extra / pr, it actually represented a sort of Weighted Throughput based on Importance Sampling.

So instead what I calculated was:

$$T_{\text{MIS Weighted}} = \frac{\beta_{\text{after}}}{\beta_{\text{before}}}$$

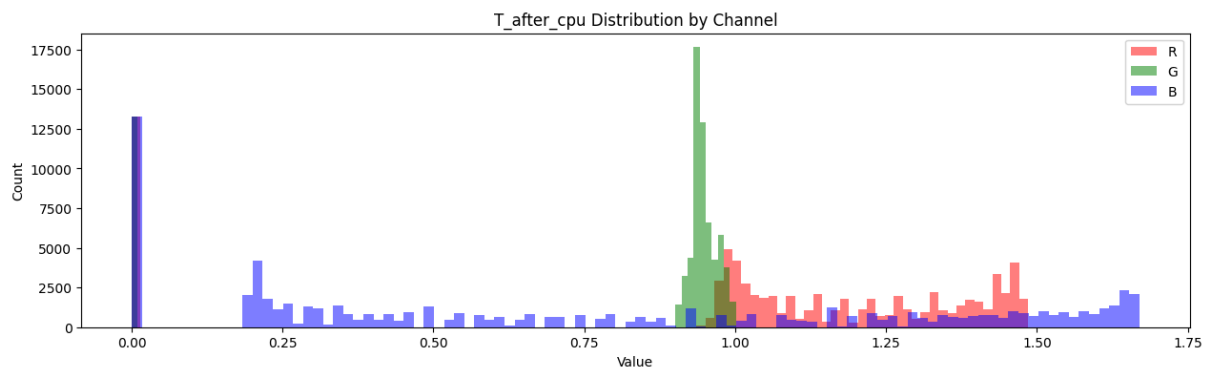
Throughput Ratio vs Pure Transmittance

Initially, I thought it predicted just a Pure transmittance, meaning it is the fraction of radiance that exits the medium from the radiance that enters. So a transmittance of 0.5 means only half the radiance that entered exits. This is also per wavelength (or per r, g, b) so some mediums would let more red light through than say blue or green.

However, from the calculation above, it actually calculates a **weighted throughput ratio** rather than pure transmittance. This ratio includes:

1. **Physical transmittance:** $e^{-\sigma \cdot d} \in [0, 1]$
2. **Importance sampling corrections:** wavelength-dependent weights that can exceed 1.0 to maintain unbiased estimation
3. **RGB color space conversion:** spectral resampling artifacts

This can be seen from the visualization of the data:



From an analysis of training data from a subset of 75,000 samples, we can see that there are many samples in which the weighted ratio is higher than 1, up to 1.6, particularly in the blue channel. This is expected for pbrt's ratio tracking integrator, where importance sampling weights correct for sampling bias.

However, PBRT only picks a single wavelength to perform to compute PDF to sample from that decides the type of scattering:

```
// Compute probabilities for each type of scattering.
Float pAbsorb = mp.sigma_a[0] / sigma_maj[0];
Float pScatter = mp.sigma_s[0] / sigma_maj[0];
Float pNull = std::max<Float>(0, 1 - pAbsorb - pScatter);
PBRT_DBG("Medium scattering probabilities: %f %f %f\n", pAbsorb,
         pScatter, pNull);

// And randomly choose one.
int mode = SampleDiscrete({ pAbsorb, pScatter, pNull }, uMode);
```

However, this would become **biased** towards the PDF of the first wavelength considered in SampledWavelengths (say 450nm). But we want to produce an unbiased estimator (to ensure energy is not created/destroyed) and would want on average for it to be unbiased/correct. Therefore, later in Listing 1 it divides by pr , which can lead to situations where the the spectral MIS portion of $T_{maj} * mp.sigma_s / pr$ is greater than 1 if for example it is very uncommon for the first wavelength to scatter but common for longer wavelengths (like red) to scatter. Because the event pr for the first wavelength is uncommon it will boost this ray's contribution to ensure it is unbiased/to compensate.

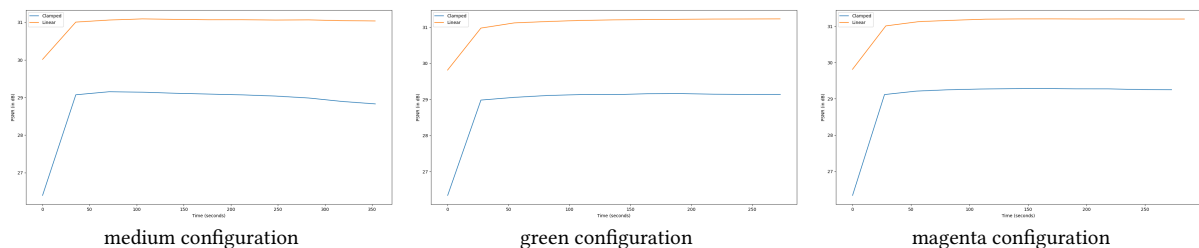
All of this is explained in [Chromatic Media](#) in the PBRT Book

I then perform a ToOutputRGB call which then converts these into RGB signals instead of the (typically 4) wavelengths according to the colourspace which by default is the CIE xyz colorspace. This boosts blue wavelengths even more helping to cause the 1.6 Transmission value as shown in the histogram above.

Results and Comparisons

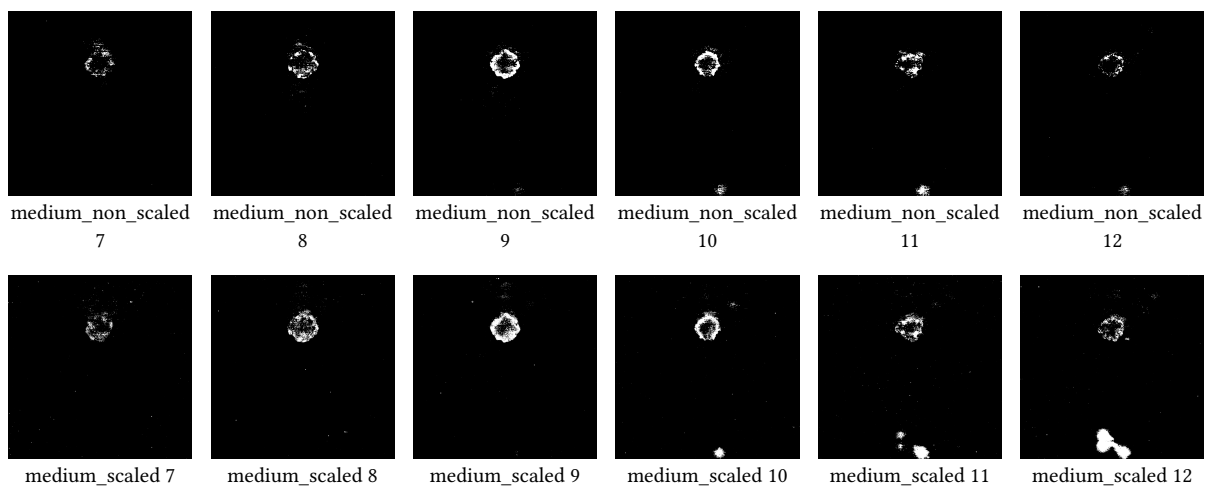
MSE and PSNR Comparison

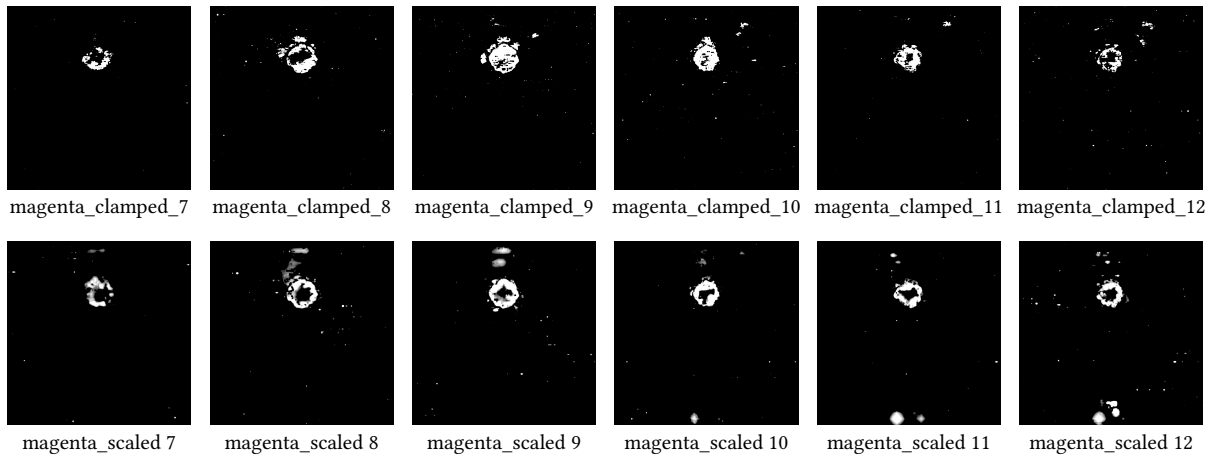
To compare with the results shown in test error MSE and PSNR metrics, using linear scaling actually provided an increase of approximately 3 dB in PSNR and a decrease of approximately 0.02 in MSE. Here are graphs showcasing the clamped and linear scaling metrics:



Slice Visualization

To compare with the slices from Visualization this is some of the results:





Though there are slight training differences (since it required retraining with the linear scaling), they still look very similar but the key thing it learned is that it doesn't have just 0/1 opacity but has a much smoother gradient/smoothier change in transmittance to get smoother transitions

What this means for the model

From the results shown above, I decided its best to pursue linear scaling for the final model/for inference rather than clamping, preserving the full distribution and also preserving the overestimates/compensation caused by the spectral MIS in PBRT. This is required to ensure an unbiased estimator in the final output image. This does require adding a new member variable `m_volume_training_tAfter_scale` to ensure the model can rescale for inference.

How this changes Inference

It doesn't really change the post calculations from the network outputs during inference:

```
Transmittance = network(p, d, tMax)
beta *= Transmittance
```

and all it would be now is

```
weighted_T = network(p, r, tMax)
```

```
beta *= weighted_T
```